

Online ensemble learning-based anomaly detection for IoT systems

Yafeng Wu, Lan Liu^{ID*}, Yongjie Yu, Guiming Chen, Junhan Hu

Guangdong Polytechnic Normal University, Guangzhou, China

ARTICLE INFO

Keywords:

Anomaly detection
Ensemble learning
Particle Swarm Optimization
IoT system
Online learning

ABSTRACT

In the modern era of digital transformation, the evolution of fifth-generation (5G) wireless networks has played a pivotal role in revolutionizing communication technology and accelerating the growth of smart technology applications. As an integral element of smart technology, the Internet of Things (IoT) grapples with the problem of limited hardware performance. Cloud and fog computing-based IoT systems offer an effective solution but often encounter concept drift issues in real-time data processing due to the dynamic and imbalanced nature of IoT environments, leading to performance degradation. In this study, we propose a novel framework for drift-adaptive ensemble learning called the Adaptive Exponentially Weighted Average Ensemble (AEWAE), which consists of three stages: IoT data preprocessing, base model learning, and online ensembling. It integrates four advanced online learning methods within an ensemble approach. The crucial parameter of the AEWAE method is fine-tuned using the Particle Swarm Optimization (PSO) technique. Experimental results on four public datasets demonstrate that AEWAE-based anomaly detection effectively detects concept drift and identifies anomalies in imbalanced IoT data streams, outperforming other baseline methods in terms of accuracy, F1 score, false alarm rate (FAR), and latency.

1. Introduction

With the rapid advancement of the fifth-generation (5G) wireless network, the Internet of Things (IoT) has emerged as a groundbreaking paradigm, facilitating a wide range of innovative features and applications [1]. The Internet of Things (IoT) has permeated a broad spectrum of everyday life, transforming various domains with its applications. This includes the development of smart cities, smart logistics [2], optimization of Intelligent Transportation Systems (ITS), advancement of healthcare, innovation in smart agriculture, and more [3]. However, IoT devices themselves possess only limited computational power, memory, and storage capacity [4]. To address these issues, cloud computing and fog computing have been proposed. Cloud computing provides the hardware resources that IoT devices lack, while fog computing resolves the distance problem between cloud computing servers and IoT devices. A typical computing structure of IoT system is shown in Fig. 1. In this setup, IoT devices transmit information to their nearest fog node via various means such as Wi-Fi, Cellular, Bluetooth, Zigbee, etc [3]. The fog nodes then perform basic processing, storage, or configuration tasks, and subsequently pass the processed information to the IoT cloud servers. These servers are responsible for analyzing the IoT data, which includes traffic monitoring tasks like anomaly detection.

Anomaly detection models are pivotal in identifying potential security threats and system malfunctions within IoT systems. Traditional

machine learning methods, when enhanced for anomaly detection, can effectively identify irregularities in IoT data streams by leveraging historical patterns and learned features to detect deviations from expected behavior.

However, the widespread use of wireless networks and the untended nature of IoT devices make these systems highly susceptible to a wide array of new attacks [5]. Moreover, factors like network configuration changes, system updates, and the introduction of new technologies contribute to unpredictable shifts in data stream distribution, a phenomenon known as concept drift [6]. Since traditional machine learning models are generally trained on static historical data, they struggle to adapt to these evolving conditions, making it challenging to effectively address the dynamic and diverse threat landscape in IoT environments.

Additionally, IoT data streams are typically imbalanced, with far more instances of normal data than attacks. Among attacks, probes scanning for network vulnerabilities are more frequent than serious threats such as botnets, firmware attacks, and privacy breaches, which are rare. This data imbalance often hinders single detection methods from effectively identifying intrusions, resulting in a higher rate of false negatives.

Online learning methods monitor data streams to identify changes and adapt to shifts in data distribution over time, maintaining relevance

* Corresponding author.

E-mail address: liulan@gpnu.edu.cn (L. Liu).

<https://doi.org/10.1016/j.asoc.2025.112931>

Received 25 April 2024; Received in revised form 1 December 2024; Accepted 21 February 2025

Available online 3 March 2025

1568-4946/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

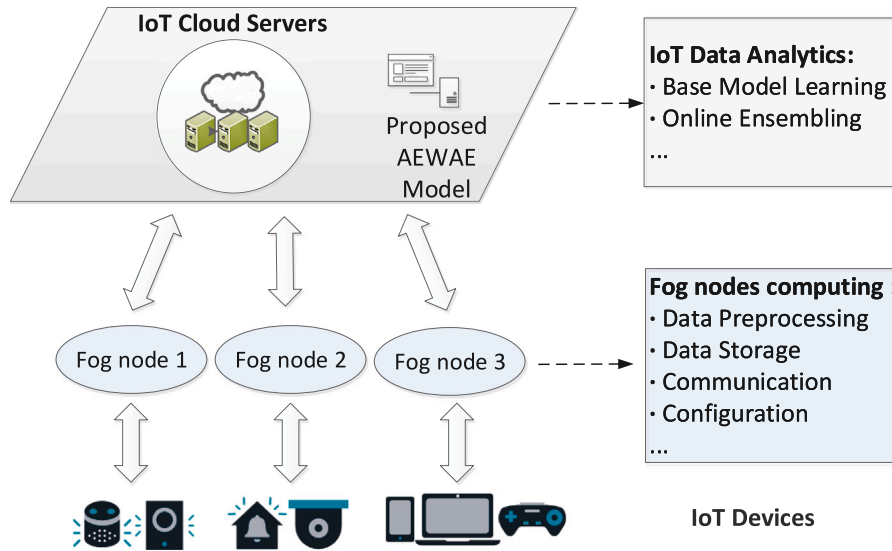


Fig. 1. The architecture of an IoT system.

as patterns evolve in real-world scenarios. By continuously updating with new data, these methods effectively mitigate the impact of concept drift in IoT data streams.

Furthermore, ensemble methods, which leverage multiple learning algorithms, aim to achieve superior predictive performance compared to any single base model. Specifically, weighted ensemble methods can assign greater importance to models that exhibit better performance on the minority class. In scenarios where new types of attacks, typically the minority class in datasets, begin to emerge more frequently, online learning models are well-suited to rapidly adapt to these changes. Therefore, a weighted ensemble algorithm that incorporates online learning methods offers a promising solution for addressing the imbalanced challenges associated with IoT data analytics.

In order to address the challenges of concept drift and data imbalance in anomaly detection for IoT systems, this study proposes a novel framework that integrates the Adaptive Exponentially Weighted Average Ensemble (AEWAE), Particle Swarm Optimization (PSO) for hyperparameter tuning, and four online learning methods. It involves three phases: IoT data preprocessing, base model learning, and online ensembling. The preprocessing component can be implemented on fog nodes, while the base model learning and online ensembling are carried out on the cloud servers, as illustrated in Fig. 1. To assess the efficacy and performance of the proposed ensemble framework, four public datasets, namely IoTID20 [7], NSL-KDD [8], IoT23 [9], and CICIoT2023 [10], are utilized for evaluation purposes.

The main contributions of this article can be summarized as follows:

- Four distinct online learning methods, ARF-ADWIN, ARF-EDDM, OPA, and KNN-ADWIN, are utilized to monitor data streams, enabling the detection of changes and adaptation to concept drift in IoT streams. Implementing these methods enhances the framework's accuracy and robustness in anomaly detection.
- A novel weighted ensemble algorithm, the Adaptive Exponentially Weighted Average Ensemble (AEWAE), is introduced to address data imbalance in IoT streams by assigning greater importance to models that perform better on the minority class.
- Particle Swarm Optimization (PSO) is employed to optimize key parameters of AEWAE, providing self-adaptive capabilities to handle varying and evolving IoT data environments.
- A streamlined ensemble framework for anomaly detection in IoT environments is proposed, incorporating ARF-ADWIN, ARF-EDDM, OPA, KNN-ADWIN, PSO, and AEWAE. This framework is

validated on four public datasets, demonstrating superior performance compared to other leading methods. The code implementation is openly accessible on <https://github.com/YafengWu/AEWAE>.

The paper is organized as follows: Section 2 provides a detailed literature review. Section 3 details the AEWAE framework and AEWAE based anomaly detection. Section 4 presents and discusses experimental results of AEWAE framework. Section 5 provides a concluding overview of the principal findings and contributions.

2. Related work

This section reviews the extant literature in two pivotal domains pertinent to our research: intrusion detection for IoT systems and online ensemble methods in anomaly detection.

2.1. Intrusion detection for IoT systems

IoT security is a critical area of focus, essential for safeguarding both the hardware and networks within IoT systems [11]. Recent scholarly contributions in this field predominantly concentrate on four key areas: supply chain security, smart home security systems, automotive security, and intrusion detection [12]. Toorajipour et al. [13] conducted a comprehensive review of key studies on the application of artificial intelligence (AI) in supply chain management (SCM), highlighting both current and potential AI techniques that could advance the field. Huang et al. [14] proposed a novel conflict taxonomy with formal definitions of various conflict types that may arise in smart home environments, providing a thorough overview of state-of-the-art conflict detection methods. Wang et al. [15] reviewed machine learning applications in the Internet of Vehicles and identified unresolved challenges and future directions in blockchain intelligence.

In the context of intrusion detection, Abusitta et al. [16] introduced a denoising autoencoder-based anomaly detection system for IoT, designed to learn and capture robust features, which are subsequently passed to a classifier. Altunay et al. [17] proposed a hybrid CNN-LSTM-based intrusion detection system tailored for industrial IoT networks. Wang et al. [18] developed a knowledge-distillation-based IoT intrusion detection model called BT-TPF, designed for IoT environments with limited computing resources. This model employs a Siamese network for feature extraction and a vision transformer as the teacher model. Sharma et al. [19] presented a deep learning framework incorporating

Generative Adversarial Networks (GANs) to handle class imbalance, with a neural network used as the classifier. Despite these advancements in intrusion detection for IoT, the challenge of poor adaptation to new data remains, highlighting the need for innovative approaches such as online learning.

2.2. Online ensemble methods in anomaly detection

Online learning methods can be classified into two primary types: drift detection methods and drift adaptation methods. Significantly, drift detection methods are essential for monitoring data streams and identifying changes. These methods can be broadly categorized into two types: data distribution-based and error rate-based [6]. Among the former, ADWIN, proposed by Bifet et al. [20], is a notable example. This method utilizes adaptive sliding windows to monitor the rate of change in the data. On the other hand, error rate-based drift detection methods include DDM [21] and EDDM [22]. DDM operates by establishing warning and drift thresholds for the error rate, which are dynamically adjusted based on the standard deviation of the error rate. Conversely, EDDM focuses on monitoring the distance between errors to detect drifts.

Drift adaptation is crucial for adjusting learning models in response to concept drift. Incremental learning techniques, such as Hoeffding Trees (HTs) [23] and Hybrid Adaptive Tree Traversal (HATT) [24], offer effective solutions for this purpose. They adaptively update their models with new data, efficiently managing concept drifts. The Adaptive Random Forest (ARF) [25], an ensemble method, combines multiple HTs to improve classification performance by leveraging collective tree predictions. In contrast, the Streaming Random Patches (SRP) algorithm [26] utilizes random subspaces and online bagging, training diverse learners on different data subsets. Different from these methods, KNN-ADWIN [27] integrates the k-Nearest Neighbors (KNN) algorithm with ADWIN, offering dynamic response to data shifts.

Ensembling different online learning methods can significantly enhance prediction accuracy and robustness. In this regard, Chiang et al. [28] proposed the Modified Deterministic Sampling Classifier with weighted Bagging (MDSCB), a chunk-based ensemble approach addressing concept drift and data imbalance. Cai et al. [29] developed a Concept Drift detection model using a Bidirectional Temporal Convolutional Network (BiTCN), combining six base learners with BiTCN for enhanced accuracy and employing a multi-stacking ensemble method. Li et al. [30] introduced the Performance Weighted Probability Averaging Ensemble (PWPAE) for drift adaptation, followed by a multistage automated network analytics framework [31] employing a window-based ensemble method for concept drift adaptation in IIoT systems. While the existing contributions are significant, none have combined exponentially weighted averaging with ensemble methods in the context of online learning. Moreover, PSO is integrated to adapt the model to various environments. As a result, an intelligent anomaly detection model based on the AEWAEE framework is employed to effectively address concept drift and data imbalance in IoT systems.

3. Proposed framework

In this section, we discuss the challenges of anomaly detection in IoT systems and provide a concise overview of the proposed AEWAEE framework. We then introduce its data preprocessing, base model learning, and online ensembling components, followed by the presentation of an advanced anomaly detection system for IoT systems based on this framework.

3.1. Problem formulation

In IoT anomaly detection, we analyze a labeled data stream $X_t = \{x_1, x_2, \dots, x_k\}$ at time t , with corresponding labels $Y_t = \{y_1, y_2, \dots, y_k\}$,

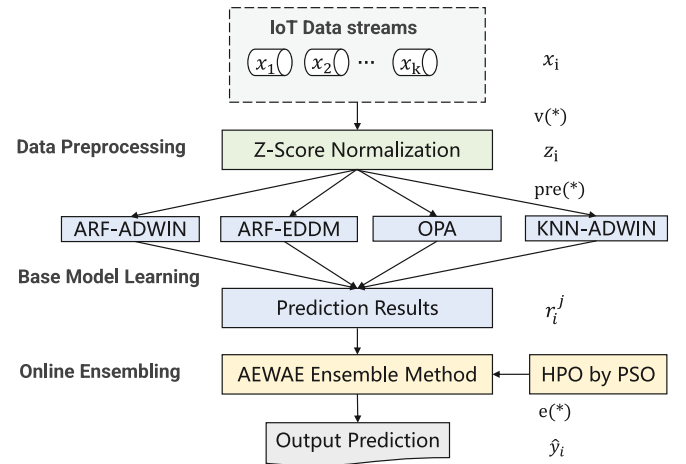


Fig. 2. The proposed AEWAEE ensemble framework.

where the dataset comprises k samples, split into m normal instances and h anomalies across q types ($h = \sum_{j=1}^q h_j$). We assume $m \gg h_j$, reflecting the significant imbalance typical in IoT data streams. Considering a subsequent labeled data stream at time $t+1$, $X_{t+1} = \{x_1, x_2, \dots, x_n\}$ and its corresponding label set $Y_{t+1} = \{y_1, y_2, \dots, y_n\}$ with n samples, we define the joint probability of the data stream X_t and its labels Y_t at time t as $p^t(X_t, Y_t)$. If there is a discrepancy such that $p^t(X_t, Y_t) \neq p^{t+1}(X_{t+1}, Y_{t+1})$, this indicates the occurrence of concept drift. Such drift may be attributed to various factors, including changes in network infrastructure or the evolution of new attack strategies.

To effectively address the challenges posed by concept drift and data imbalance, we have proposed the AEWAEE framework, as illustrated in Fig. 2. This framework is composed of three integral components: data preprocessing, base model learning, and online ensembling. For the pre-processed data stream, our approach employs four foundational models to execute predictive analyses. Subsequently, we leverage the AEWAEE method to aggregate their predictions into a cohesive ensemble. This methodology is designed to enhance the accuracy and reliability of the predictive outcomes.

3.2. Overview of AEWAEE framework

Data preprocessing enhances model training, ensuring robustness, adaptability, and efficiency in online learning methods. For a given data point x_i within the data stream $X_t = \{x_1, x_2, \dots, x_k\}$, the preprocessing method transforms the original data point x_i into a preprocessed value z_i , which is computed as follows:

$$z_i = v(x_i, \theta_v) \quad (1)$$

where θ_v represents the parameters of the preprocessing method.

To proficiently detect and adapt to concept drift in our data, we employ a dual strategy encompassing two distinct drift detectors, namely EDDM and ADWIN, and two divergent drift adaptation methodologies: ARF and KNN-ADWIN. In contrast to these reactive strategies, our approach also incorporates the OPA algorithm, a method that uniquely adapts to incoming data in real-time. Consequently, we employ a robust suite of four methodologies, including ARF-EDDM, ARF-ADWIN, OPA, and KNN-ADWIN, formulated as m_1, m_2, m_3, m_4 , to facilitate predictions based on the normalized data point z_i , as follows:

$$r_i^j = \text{pre}(z_i, \theta_j) \quad (2)$$

where $j \in \{m_1, m_2, m_3, m_4\}$, and θ_j denotes the relevant parameters associated with that method j .

To enhance the predictive outcomes' accuracy and reliability, we employ the AEWAEE ensemble method, which aggregates predictions

from four distinct online learning methods. The ensemble formula is given by:

$$\hat{y}_i = e(r_i^{m1}, r_i^{m2}, r_i^{m3}, r_i^{m4}, \theta_e) \quad (3)$$

where θ_e denotes the parameters specific to the AEWA ensemble method.

3.3. Data preprocessing

Data from different IoT devices have varying ranges and units, so normalization is employed to ensure that all features contribute equally to model training. In this work, we use the Z-score normalization technique, which is calculated as follows:

$$z_i = \frac{x_i - \mu_t}{\sigma_t} \quad (4)$$

where μ_t and σ_t represent the mean and standard deviation of the data stream X_t , respectively.

3.4. Base model learning

Drift detection. In our framework, we incorporate two detection methods, EDDM [22] and ADWIN [20], to identify concept drift. The EDDM approach is particularly adept at monitoring gradual drift. It achieves this by examining the average distance between consecutive classification errors, which acts as an indicator of potential drifts.

To illustrate how EDDM functions, let us consider a data stream $X_t = \{x_1, x_2, \dots, x_k\}$, where errors occur at points $X_{\text{error}} = \{e_1, e_2, \dots, e_a\}$. The distances between these error points are represented by $D_{\text{error}} = \{d_1, d_2, \dots, d_{a-1}\}$. The EDDM method then calculates the average p_i for each error point as follows:

$$p_i = \frac{d_1 + d_2 + \dots + d_i}{i} \quad (5)$$

where $i \in \{1, 2, \dots, a-1\}$.

Algorithm 1 ADWIN Drift Detection Method.

Input: Data stream $X_t = \{x_1, x_2, \dots, x_k\}$, relevant bucket stream $B_t = \{b_1, b_2, \dots, b_c\}$, window W , mean values of subwindows μ_0 and μ_1 , threshold ε .

Output: Drift detection in the data stream.

```

1: for  $b_i$  in  $B_t$  do
2:    $W \leftarrow W \cup \{b_i\}$ ;
3:   for  $b_j$  in  $W$  do
4:      $W_0 \leftarrow W[0 : b_j]$ ;
5:      $W_1 \leftarrow W[b_{j+1} : \text{end}]$ ;
6:     if  $|\mu_0 - \mu_1| > \varepsilon$  then drop  $W_0$ ;
7:   end if
8: end for
9: end for
10: return  $W$ 

```

For example, In the given data stream, denoted as $X_t = \{1, 2, 3, \dots, 50\}$, errors are identified at specific points, labeled as $X_{\text{error}} = \{3, 8, 15, 20, 30, 40, 50\}$. The intervals between these errors, represented as $D_{\text{error}} = \{5, 7, 5, 10, 10, 10\}$. Based on these intervals, the sequentially calculated average distances, expressed as P_i , are $\{5, 6, 5.67, 6.75, 7.4, 7.83\}$.

To further enhance the detection precision, the standard deviation of p_i , denoted as s_i , is computed using the following equation:

$$s_i = \sqrt{\frac{p_i(1 - p_i)}{i}} \quad (6)$$

Finally, the EDDM drift detection method is defined as follows:

$$\begin{cases} \text{if } (p_i + 2 \cdot s_i) / (p_{\max} + 2 \cdot s_{\max}) < \alpha \rightarrow \text{Warning} \\ \text{if } (p_i + 2 \cdot s_i) / (p_{\max} + 2 \cdot s_{\max}) < \beta \rightarrow \text{Drift} \end{cases} \quad (7)$$

where α and β represent the thresholds for warning and drift, respectively, and $\alpha > \beta$. Here, $p_{\max}$ denotes the maximum average error distance, and $s_{\max}$ signifies the maximum standard deviation of p_i .

In our approach, we utilize the ADWIN algorithm as another method for drift detection, which operates by comparing mean values within two distinct, automatically adjusting sliding windows. This process is elucidated in Algorithm 1. Consider a data stream X_t that is divided into distinct buckets B_t . Over this data stream, a dynamically sized sliding window W is maintained. This window W is further divided into two sub-windows: W_0 and W_1 , with W_0 encompassing the older data and W_1 containing the most recent data. Let μ_0 and μ_1 represent the mean values of windows W_0 and W_1 , respectively. A drift is detected if the absolute difference between these means exceeds a certain threshold, specifically $|\mu_0 - \mu_1| > \varepsilon$. Upon detecting a drift, the algorithm discards window W_0 . If no drift is detected, window W is updated to incorporate new data. The threshold ε is defined as follows:

$$\varepsilon = \sqrt{\frac{2 \cdot s \cdot \ln(\frac{2 \cdot \ln(w)}{\delta})}{n}} + \frac{2 \cdot \ln(\frac{2 \cdot \ln(w)}{\delta})}{3 \cdot n} \quad (8)$$

where s represents the variance of the data within the window W , and w denotes the size of this window. The parameter δ is a constant that establishes the confidence level for change detection within the data stream. The term n is a crucial parameter in the computation of ε and is defined as:

$$n = \frac{1}{n_0} + \frac{1}{n_1} \quad (9)$$

where n_0 and n_1 signify the number of elements in two subwindows W_0 and W_1 respectively, which together constitute the window W . This formulation of n ensures a balance between the sizes of the subwindows, contributing to the robustness of the change detection process.

Drift adaptation. ARF is an advanced methodology that synergizes Hoeffding Trees with online bagging. As outlined in Algorithm 2, ARF effectively adapts decision trees to data streams with potential concept drift. Initially, it sets up n trees and corresponding weights. For each data point in the stream, the algorithm iterates through each tree to make predictions, update weights based on these predictions, and trains the trees with the new data, using up to m features per split. To address concept drift, it creates and trains backup trees upon detecting a warning signal. If drift is later detected, these backups replace the original trees, ensuring the model swiftly adapts to new data patterns.

Transitioning from tree-based methods, the KNN algorithm presents an alternative yet comparably effective approach. When integrated within the adaptive window W of the ADWIN algorithm, KNN adeptly manages individual data points x_i in a data stream $X_t = \{x_1, x_2, \dots, x_k\}$. Predicated on the principle that similar entities are often in close proximity, KNN categorizes a new data point by employing a majority voting mechanism among its k nearest neighbors. The distance between data points is calculated using Euclidean distance. The assignment of the class label, r_i^{m4} , for a new data point is formulated as follows:

$$r_i^{m4} = \underset{c \in C}{\text{argmax}} \text{count}(c, N_k(x_i)) \quad (10)$$

where C represents the set of possible classes, $N_k(x_i)$ denotes the set of k nearest neighbors of the data point x_i , and $\text{count}(c, N_k(x_i))$ calculates the frequency of class c within this set.

In our framework, we distinguish from traditional reactive drift detection methods by incorporating the Online Passive-Aggressive (OPA) algorithm. This algorithm can adapt to incoming data in real-time. In the given data stream, which consists of instances denoted as (x_i, y_i) within the sets X_t and Y_t , the values of y_i are transformed into the binary set $\{-1, 1\}$. We initiate with a weight vector w where $w_0 = 0$. For each instance x_i in the stream, the OPA algorithm predicts as follows:

$$r_i^{m3} = \text{sign}(w_i \cdot x_i) \quad (11)$$

where w_i represents the weight vector of instance x_i . The $\text{sign}(\ast)$ is a mathematical function that extracts the sign of a real number, which is

Algorithm 2 ARF Drift Adaptation Method

Input: Data stream $X = \{x_1, x_2, \dots, x_k\}$, relevant label set $Y = \{y_1, y_2, \dots, y_k\}$, number of trees n , maximum features per split m .
Output: Properly adapted trees for the data stream with drift adaptation.

```

1:  $T \leftarrow \text{CreateTrees}(n);$   $\triangleright$  Initialize  $n$  trees
2:  $W \leftarrow \text{InitWeights}(n);$   $\triangleright$  Initialize weights for each tree
3: for  $x_i$  in  $X$  do  $\triangleright$  Iterate through each data point
4:   for  $t$  in  $T$  do  $\triangleright$  Iterate through each tree
5:      $r_i^t \leftarrow \text{Predict}(t, x_i);$   $\triangleright$  Make a prediction for  $x_i$ 
6:      $W_t \leftarrow \text{UpdateWeights}(W_t, r_i^t, y_i);$   $\triangleright$  Update weights based on prediction
7:      $\text{TrainTree}(t, m, x_i, y_i);$   $\triangleright$  Train the tree with the data point
8:     if  $\text{WarningDetected}()$  then
9:        $b \leftarrow \text{CreateTree}();$ 
10:       $B(t) \leftarrow b;$   $\triangleright$  Create a backup tree on warning
11:     end if
12:     if  $\text{DriftDetected}()$  then
13:        $t \leftarrow B(t);$   $\triangleright$  Replace tree with backup on drift
14:     end if
15:   end for
16:   for  $b$  in  $B$  do
17:      $\text{TrainTree}(b, m, x_i, y_i);$   $\triangleright$  Train background trees
18:   end for
19: end for
20: return  $T$   $\triangleright$  Return the adapted trees
```

Algorithm 3 Base Model Learning

Input: Data stream $X_t = \{x_1, x_2, \dots, x_k\}$, corresponding label set $Y_t = \{y_1, y_2, \dots, y_k\}$.
Output: Prediction results r_i^j .

```

1: for each pair  $(x_i, y_i)$  in  $X_t, Y_t$  do
2:   Detect concept drift using Eq. (7). Adapt and predict  $r_i^{m1}$  using Algorithm 2;  $\triangleright$  ARF-EDDM
3:   Detect concept drift using Algorithm 1. Adapt and predict  $r_i^{m2}$  using Algorithm 2;  $\triangleright$  ARF-ADWIN
4:   Predict  $r_i^{m3}$  using Eq. (11). Detect concept drift using Eq. (13) and adapt using Eq. (14);  $\triangleright$  OPA
5:   Detect and adapt to concept drift using Algorithm 1. Predict  $r_i^{m4}$  using Eq. (10);  $\triangleright$  KNN-ADWIN
6: end for
7: return  $r_i^{m1}, r_i^{m2}, r_i^{m3}, r_i^{m4}$ 
```

defined as follows:

$$\text{sign}(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & \text{if } x = 0 \\ -1, & \text{if } x < 0 \end{cases} \quad (12)$$

Subsequently, we compute the loss for this prediction. The loss is zero if the prediction aligns with the true label; otherwise, it is calculated based on the margin, defined as:

$$\mathcal{L}_i = \max(0, 1 - y_i \cdot r_i^{m3}) \quad (13)$$

where y_i is the true label of the instance x_i .

In cases where the loss exceeds zero, the algorithm updates the weights more aggressively, following the strategy:

$$\mathbf{w}_{i+1} = \mathbf{w}_i + \tau_i \cdot y_i \cdot x_i \quad (14)$$

where τ_i denotes a dynamically computed value analogous to a learning rate. It determines the magnitude of the weight update at each step and

is defined as:

$$\tau_i = \min(C, \frac{\mathcal{L}_i}{\|x_i\|^2}) \quad (15)$$

where $\|x_i\|^2$ refers to the squared norm of the vector x_i and C is a constant. This adaptive nature of τ_i ensures that the model remains responsive to changes in the data stream, thereby enhancing the accuracy of real-time predictions.

As a result, the final online base models selected for the proposed framework are ARF-EDDM, ARF-ADWIN, OPA, and KNN-ADWIN. The overall learning process is outlined in Algorithm 3. The reasons for selecting these as foundational models are detailed below:

(1) ADWIN and EDDM exhibit proficiency in handling both gradual and abrupt concept drifts. The concurrent utilization of these methods facilitates the comprehensive detection of diverse forms of concept drift, enhancing the adaptability and robustness of the system.

(2) ARF-ADWIN and ARF-EDDM, both extensions of the Adaptive Random Forest (ARF) method, have demonstrated improved performance in handling concept drift when compared to alternative drift adaptation approaches, as evidenced by experimental results conducted in [25]. The use of these two variants in the ensemble model allows the proposed framework to benefit from their enhanced adaptability to concept drift, further improving the overall performance of the system.

(3) One of the key advantages of the selected models, ARF-ADWIN, ARF-EDDM, OPA and KNN-ADWIN, is their ability to deliver strong performance while maintaining shorter execution times. This balance between efficiency and effectiveness makes them well-suited for real-time IoT data stream analysis and handling concept drift.

(4) To optimize performance, an ensemble model must uphold considerable heterogeneity among its base learners, thereby enhancing the probability of elevating its learning outcomes. Despite using different learning algorithms, the chosen models, ARF, OPA, and KNN-ADWIN, exhibit sufficient diversity in their construction processes to result in a more resilient and varied ensemble framework.

3.5. Online ensembling

Through the previous training of base learners, their classification results r_i^j are obtained. Next, a new ensemble method called AEWAEE will be used to enhance performance.

AEWAEE refers to an approach that utilizes a weighted average approach, where probabilities of each component model are assigned a weight, which is multiplied by the prediction probability of the component model, and then the weights are added to obtain the final prediction probability. The final predicted class is the one with the highest average probability. Consider data point (x_i, y_i) within the sets X_t and Y_t , the ensemble prediction $\hat{y}_i$ for each input x_i is computed using a weighted sum of probabilities from multiple models. Formally, the prediction is given by:

$$\hat{y}_i = \underset{k \in \{0,1\}}{\text{argmax}} \sum_{j=1}^{m4} (w_i^j \cdot p_j(r_i^j = k \mid \theta_j, x_i)) \quad (16)$$

where w_i^j represents the weight assigned to the j model in the ensemble, r_i^j denotes the classification result of data point x_i using the j model, and $p_j(r_i^j = k \mid \theta_j, x_i)$ indicates the probability that the j model assigns to class k for the input x_i , given the learned parameters θ_j .

The computation of the weight w_i^j for individual foundational learner j is derived from the inverse of the learner's current error rate, as discussed below:

$$w_i^j = \frac{1/(\text{ewa_err}_i^j + \epsilon)}{\sum_{k=1}^4 1/(\text{ewa_err}_i^k + \epsilon)} \quad (17)$$

where ϵ is a regularization parameter introduced to prevent the occurrence of a zero denominator. This setting allows for higher weights to be assigned to lower error rates, thereby helping to improve the performance of the model.

The error rate for each foundational learner, denoted as $ewa_err_i^j$, can be quantitatively assessed using the Exponentially Weighted Average (EWA) method, calculated as follows

$$ewa_err_i^j = \frac{cum_err_i^j}{1 - \alpha^{(i+1)}} \quad (18)$$

where α indicates the weighting decay factor for historical errors when calculating the exponentially weighted moving average.

Dividing by $(1 - \alpha^{(i+1)})$ is to normalize the cumulative error, so that $ewa_err_i^j$ is scaled to the range between 0 and 1. This scaling is necessary to make $ewa_err_i^j$ comparable across different iterations and samples, and to enable it to serve as a reliable indicator of the algorithm's prediction accuracy.

The cumulative error rate attributed to each foundational learner can be denoted by

$$cum_err_i^j = \alpha \cdot cum_err_{i-1}^j + (1 - \alpha) \cdot err_i^j \quad (19)$$

where $cum_err_{i-1}^j$ represents the cumulative error of previous item x_{i-1} , and α is a weighting factor or a smoothing parameter that determines the contribution of the previous cumulative error to the current value of the cumulative error.

Algorithm 4 Online Ensembling

Input: Data stream $X_t = \{x_1, x_2, \dots, x_k\}$, corresponding label set $Y_t = \{y_1, y_2, \dots, y_k\}$.

Output: Ensemble predictions $\hat{y}_i$.

```

1: for each pair  $(x_i, y_i)$  in  $X_t, Y_t$  do
2:   for  $j = m1$  to  $m4$  do
3:     Obtain prediction result  $r_i^j$  using Algorithm 3;
4:     Calculate the error rate  $err_i^j$  using Eq. (20);
5:     Compute the cumulative error rate  $cum\_err_i^j$  using Eq. (19);
6:     Determine the exponentially weighted average error rate  $ewa\_err_i^j$  using Eq. (18);
7:     Calculate the weight  $w_i^j$  using Eq. (17);
8:   end for
9:   Determine the ensemble prediction  $\hat{y}_i$  using Eq. (16);
10: end for
11: return  $\hat{y}_i$ 

```

Specifically, α is a constant between 0 and 1. When α approaches 1, the decay is slower, and the older cumulative error, denoted as cum_err , becomes more significant in the update process. The model thus places greater emphasis on past historical error information and is capable of observing a greater number of historical samples. Conversely, as α approaches 0, the decay accelerates, and the importance of the new prediction error, labeled as err , increases in the update process. This condition suggests that the model favors more recent error information and only pays attention to a minimal number of historical samples.

The cumulative error rate is a variation that draws inspiration from the exponentially weighted average calculation method. Exponentially Weighted Moving Average (EWMA) charts were initially proposed in [32] for detecting an increase in the mean of a sequence of random variables. The study began to consider real-time dynamic control of processes using discrete data [33]. By employing this method to calculate the weight, it can further enhance our adaptability to dynamic IoT data streams.

The error rate associated with each base learner, can be expressed as follows:

$$err_i^j = \begin{cases} 1, & \text{if } (r_i^j \neq y_i) \\ 0, & \text{otherwise} \end{cases} \quad (20)$$

where r_i^j denotes the predicted label from the j base model; and y_i represents the actual label for the data point x_i .

Consequently, in the AEWA method, as outlined in Algorithm 4, the process begins by calculating the error rate err_i^j for each base learner based on its prediction result r_i^j . This error rate is then used

Algorithm 5 Hyperparameter Optimization (HPO)

Input: Data stream X_t , corresponding label set Y_t , $Space$: hyperparameter configuration space, $MaxTime$: the maximum parameter search times.

Output: $MaxAcc$: the best accuracy, HP_{opt} : optimal hyperparameter values.

```

1: function HPO( $X_t, Y_t, Space, MaxTime$ )
2:    $MaxAcc \leftarrow 0$ ;
3:   for  $n = 1$  to  $MaxTime$  do
4:      $\alpha \leftarrow \alpha_i^{new}$ ; ▷ Calculated by Eq. (21)
5:      $Acc \leftarrow AEWA(X_t, Y_t, \alpha)$ ; ▷ Evaluate the present  $\alpha$  value
6:     if  $MaxAcc < Acc$  then
7:        $MaxAcc \leftarrow Acc$ ; ▷ Update accuracy
8:        $HP_{opt} \leftarrow \alpha$ ; ▷ Update the optimal  $\alpha$  value
9:     end if
10:  end for
11:  return  $MaxAcc, HP_{opt}$  ▷ The best accuracy and  $\alpha$ 
12: end function

```

to determine the cumulative error rate $cum_err_i^j$ and the exponentially weighted average error rate $ewa_err_i^j$. Subsequently, these metrics inform the computation of the relative weight w_i^j for the j base learner. The ensemble prediction is ultimately derived by applying these weights.

The AEWA model exhibits dynamic adaptability to diverse scenarios by leveraging the optimized α to make ensemble method adjustments. Consequently, Particle Swarm Optimization is employed to fine-tune this hyperparameter in order to achieve an optimized ensemble model.

Particle Swarm Optimization is a meta-heuristic optimization technique that simulates the collaborative and knowledge-sharing behavior of entities within a swarm [34]. This approach aids in efficiently searching for optimal solutions due to its low computational complexity and capability for parallel execution. The update strategy for a particle's value α , denoted as α_i^{new} , in the swarm is governed by the following equation:

$$\alpha_i^{new} = \alpha_i + v_i^{new} \quad (21)$$

where α_i represents the current value of particle i , and v_i^{new} denotes the newly updated velocity of the same particle, which is formulated as:

$$v_i^{new} = w \cdot v_i + c_1 \cdot \text{ran1}() \cdot (pbest_i - \alpha_i) + c_2 \cdot \text{ran2}() \cdot (gbest - \alpha_i) \quad (22)$$

where v_i is the current velocity of particle i , $pbest_i$ is the personal best value attained by particle i , and $gbest$ is the global best value identified among all particles. The parameter w represents the inertia weight, which influences the magnitude of the particle's previous velocity. The coefficients c_1 and c_2 are cognitive and social coefficients, respectively, guiding the particle's movement towards its personal best and the global best values. The functions $\text{ran1}()$ and $\text{ran2}()$ generate random numbers between 0 and 1, introducing stochastic elements to the velocity update.

Algorithm 6 AEWA Enhanced Anomaly Detection

Input: Data stream $X_t = \{x_1, x_2, \dots, x_k\}$, corresponding label set $Y_t = \{y_1, y_2, \dots, y_k\}$.

Output: A trained anomaly detection model M .

```

1: Initialize the model  $M$ ;
2: for each pair  $(x_i, y_i)$  in  $X_t, Y_t$  do
3:   Normalize the data point  $x_i$  using Eq. (4);
4:   Optimize the hyperparameter  $\alpha$  of AEWA using Algorithm 5;
5:   Train model  $M$  using Algorithm 4;
6: end for
7: return  $M$ 

```

As depicted in Algorithm 5, the execution of AEWAE requires the provision of the hyperparameter configuration space to the 'HPO' function. Subsequently, PSO identifies the most suitable hyperparameter that yields the maximum overall accuracy. The discovered optimal hyperparameter is then supplied to the AEWAE algorithm, which constructs the optimized model for precise IoT data analysis.

3.6. Anomaly detection based on AEWAE

As delineated in Algorithm 6, the AEWAE enhanced anomaly detection encompasses three distinct phases. Initially, the process involves the normalization of each data point x_i in the stream, employing Z-score normalization for standardization. Subsequently, PSO method is deployed to fine-tune the hyperparameter α of the AEWAE framework. In the final phase, the AEWAE method, now equipped with the optimized α , is applied to execute classification tasks on the normalized data points.

4. Performance evaluation

In this section, we describe experiments conducted on four public datasets to assess the performance of our proposed AEWAE framework in comparison with other state-of-the-art models.

4.1. Datasets

The effectiveness of the proposed approach is evaluated using four public security datasets: IoTID20 [7], NSL-KDD [8], IoT23 [9], and CICIOT2023 [10]. The NSL-KDD dataset simulates data transmission between servers, IoTID20 represents data generated by IoT devices, while IoT23 and CICIOT2023 provide more recent IoT threat data, reflecting evolving IoT environments. All datasets are imbalanced, posing challenges typical of real-world scenarios.

The NSL-KDD dataset includes a diverse range of attack types, such as Denial of Service (DoS), User to Root (U2R), Remote to Local (R2L), and Probing attacks, making it a valuable benchmark for evaluating intrusion detection systems (IDS) across various cybersecurity threats. It has been extensively used in research to simulate traditional network attacks and assess IDS performance.

In contrast, IoTID20 focuses on IoT security, providing a dataset of samples from various IoT devices under normal and attack conditions. It captures IoT-specific threats, such as data integrity attacks and botnets, reflecting the unique security challenges in IoT environments. Together, NSL-KDD and IoTID20 offer a comprehensive platform for studying both traditional network and IoT security issues.

To complement these, the IoT23 and CICIOT2023 datasets offer more recent samples from various IoT environments. IoT23 contains labeled IoT traffic under normal and attack conditions, reflecting botnets, including Mirai, and other evolving threats. CICIOT2023 is a large-scale dataset that captures modern IoT vulnerabilities, focusing on privacy attacks such as MitM, password attacks, browser hijacking, backdoor malware, XSS, SQL injection, and DNS spoofing.

Anomaly detection systems aim to distinguish between cyberattacks and normal system states. To evaluate our model, we use subsets of the IoTID20, NSL-KDD, CICIOT2023, and IoT23 datasets. The IoTID20 subset, sampled as in [30], contains 6,252 binary-labeled records (5,891 normal, 361 attacks) with 32 features. The NSL-KDD, CICIOT2023, and IoT23 subsets are randomly sampled from their original datasets, containing 35,141 (16,466 normal, 18,675 attacks), 15,444 (10,000 normal, 5,444 attacks), and 41,663 (20,000 normal, 21,663 attacks) records, respectively, with 42, 47, and 28 features. Before applying the AEWAE framework, we remove duplicates, fill in missing values with zeros, and label non-numeric features.

4.2. Experimental setup

The proposed framework has been realized by leveraging the Python

3.9 programming language, implemented within the Google Colaboratory environment. This was accomplished through an extension of the River library [35]. The experimental setup included a system powered by an Intel(R) Xeon(R) CPU @2.20 GHz, and supplemented with 12.7 GB of RAM, which is representative of a typical Internet of Things (IoT) core processing unit used for comprehensively analyzing data.

Two evaluation methods, cross-validation and prequential validation, are utilized to assess the proposed model. In the cross-validation strategy, one fold (10% of the dataset) is used for training, while the remaining nine folds (90% of the dataset) are reserved for testing in each iteration. On the other hand, prequential validation is a model validation method commonly employed for time-series or streaming data. It sequentially tests then trains on each data point to ensure the model never uses future data for current prediction, thereby preserving chronological order, and continually refining and updating the model with the introduction of new data.

To facilitate a comprehensive evaluation of our model's performance, we use three key metrics: accuracy, the F1-score and FAR. Meanwhile, assessing the learning speed involves measuring aspects such as latency and the cumulative distribution function.

Latency, in this context, refers to the time delay experienced when the system responds to a particular input, essentially denoting the average time our model requires to process each sample in IoT data analysis. FAR, conversely, quantifies the frequency of incorrect alerts generated by the system over a specified period. Particularly in the context of anomaly detection, it refers to the proportion of normal events mistakenly identified as anomalies by our model.

Lastly, the cumulative distribution function, a crucial statistical metric, outlines the probability that a random variable will take a value less than or equal to a specified value. Herein, it serves as a tool to measure the proportion of total learning tasks that our model can complete within a specified time range. This comprehensive evaluation approach equips us with a thorough understanding of the model's potential and the ability to analyze IoT data.

4.3. Performance of online learning method

To evaluate the comparative effectiveness of online learning methods against traditional machine learning approaches, we applied the Random Forest (RF) and ARF-EDDM to the NSL-KDD dataset. ARF-EDDM is a specialized form of RF designed to handle dynamic, evolving data streams by incorporating a drift detection method and adaptability to concept drift. Visual comparison results are illustrated in Fig. 3.

As depicted in Fig. 3(a), the RF model maintains high accuracy for a small number of samples, after which there is a sharp drop. The accuracy then gradually declines as the number of samples increases, ultimately achieving an accuracy of 83.92%. Conversely, Fig. 3(b) demonstrates a similar trend for ARF-EDDM, with the accuracy beginning to decrease at the same point. However, this line stays relatively stable even as the number of samples increases, resulting in a final accuracy of 96.55%. Notably, the EDDM algorithm within ARF-EDDM detected several instances of concept drift throughout this process.

This comparative analysis underscores the occurrence of concept drift in IoT data streams and the superior performance of the online learning method ARF-EDDM over the traditional RF approach. The key advantage of ARF-EDDM lies in its ability to efficiently detect shifts in model performance with EDDM. This capability enables the adaptive ARF algorithm to effectively respond to performance degradations, thereby enhancing overall model effectiveness.

4.4. Evaluation of anomaly detection performance

To comprehensively assess the performance of the AEWAE-enhanced anomaly detection approach, we conducted experiments focusing on accuracy, F1 score, FAR, and latency. These experiments were carried out on two imbalanced datasets and compared with other

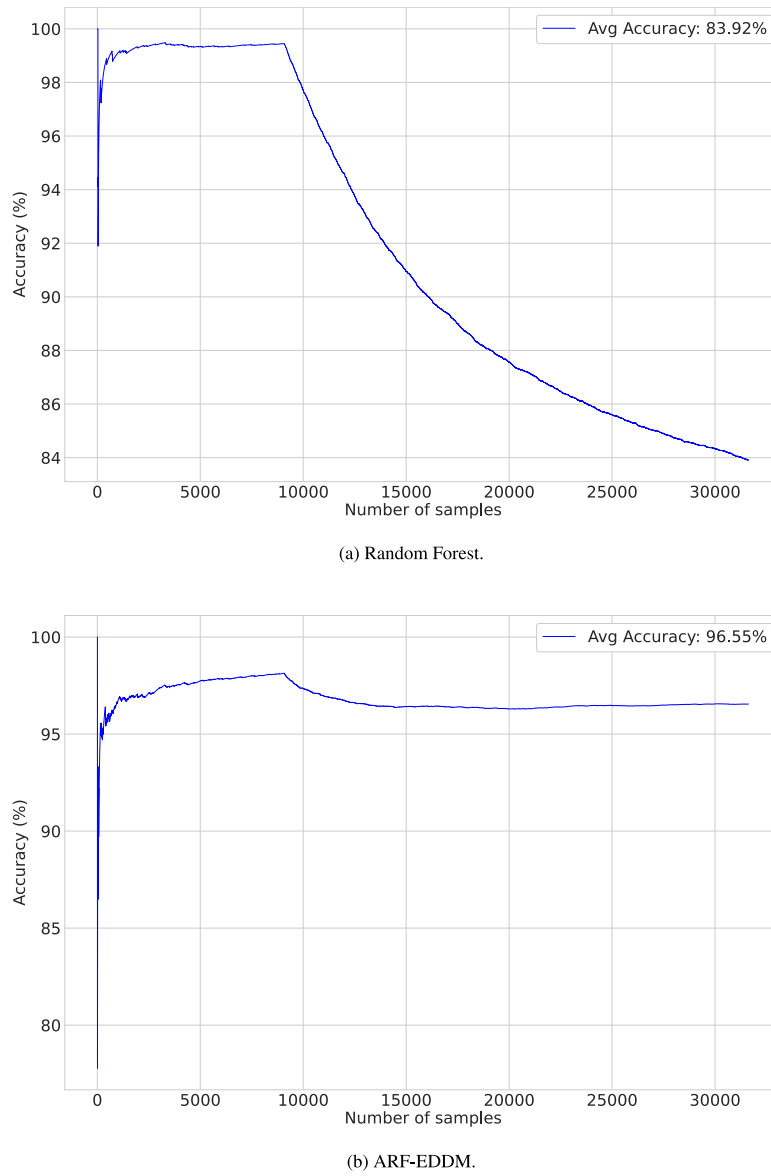


Fig. 3. Evaluation of RF and ARF-EDDM performance on the NSL-KDD dataset.

state-of-the-art online learning strategies as discussed in Section 2. The comparative methods include ARF-ADWIN [25], ARF-EDDM [25], OPA [36], SRP [26], KNN-ADWIN [27], HT [23], HATT [24], PWP-PAE [30], and MSANA [31]. Results are detailed in Figs. 4, 5 and Tables 1, 2.

As illustrated in Figs. 4 and 5, as well as Table 1 and 2, the four base models, ARF-ADWIN, ARF-EDDM, KNN-ADWIN, and OPA, achieve high accuracy while maintaining quite low latency on the NSL-KDD and IoTID20 datasets. This illustrates the reasons for choosing them as base models.

In the study of the NSL-KDD dataset, as displayed in Fig. 4 and Table 1, we encountered four instances of concept drifts throughout the experiments. The proposed AEWAE outperforms all other evaluated online learning models when the α is set to 0.006, achieving the highest accuracy (99.20%) and F1-score (99.24%). In addition, the proposed AEWAE method stands out for its swift inference time, clocking in at just 3.3 ms per packet, which significantly outpaces other ensemble techniques like PWP-PAE and MSANA. The FAR, another crucial metric of the AEWAE model, also holds commendably low at 0.88%.

Table 1

Evaluation of online learning techniques on NSL-KDD dataset. The $\pm$ values represent the standard deviation of each metric.

NSL-KDD dataset				
Method	Accuracy (%)	F1 (%)	FAR (%)	Latency (ms)
ARF-ADWIN	95.72 $\pm_{.30}$	95.98 $\pm_{.28}$	4.81 $\pm_{.48}$	0.97
ARF-EDDM	96.33 $\pm_{.24}$	96.55 $\pm_{.23}$	3.95 $\pm_{.22}$	1.31
OPA	89.47 $\pm_{.07}$	90.02 $\pm_{.07}$	10.47 $\pm_{.11}$	0.13
SRP	96.03 $\pm_{.29}$	96.27 $\pm_{.28}$	4.45 $\pm_{.36}$	5.35
KNN-ADWIN	89.48 $\pm_{.06}$	89.68 $\pm_{.06}$	6.56 $\pm_{.08}$	0.5
HT	89.36 $\pm_{.59}$	89.57 $\pm_{.59}$	6.74 $\pm_{1.29}$	0.53
HATT	90.53 $\pm_{.71}$	90.83 $\pm_{.78}$	6.94 $\pm_{1.23}$	1.4
PWP-PAE	97.48 $\pm_{.14}$	97.63 $\pm_{.14}$	2.67 $\pm_{.21}$	13.77
MSANA	95.45 $\pm_{.43}$	95.72 $\pm_{.41}$	4.60 $\pm_{.39}$	4.44
AEWAE	99.20$\pm_{.07}$	99.24$\pm_{.06}$	0.88$\pm_{.10}$	3.3

Fig. 5 and Table 2 illustrate the performance of online learning algorithms on the IoTID20 dataset, revealing the occurrence of three subtle drifts during the initial phases of the experiment. The results demonstrate that the proposed AEWAE model excels in comparison to

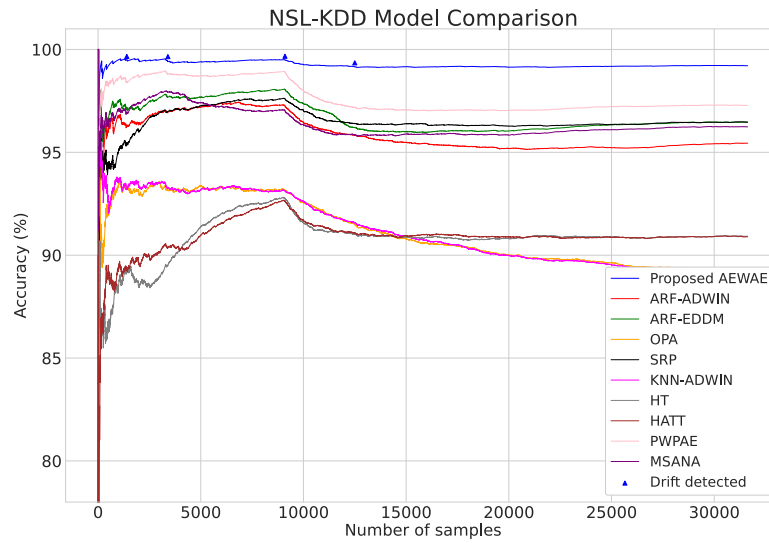


Fig. 4. Comparative analysis of accuracy in leading online learning techniques on the NSL-KDD dataset.

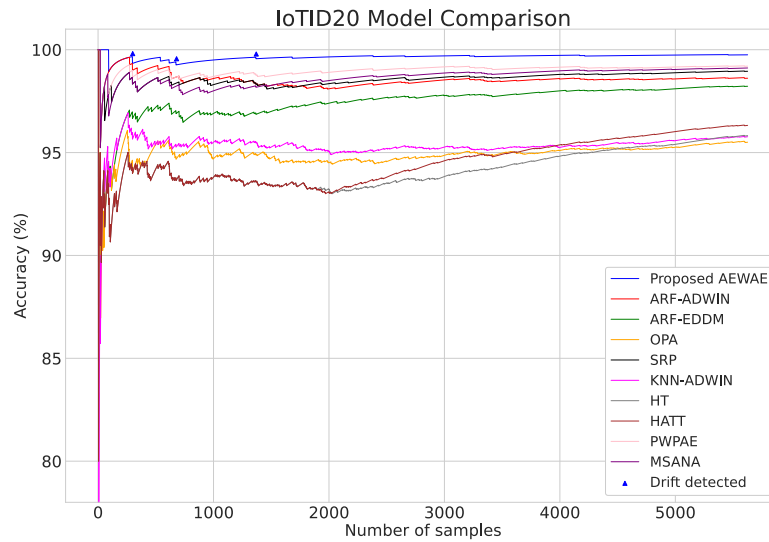


Fig. 5. Comparative analysis of accuracy in leading online learning techniques on the IoTID20 dataset.

Table 2

Evaluation of online learning techniques on IoTID20 dataset. The $\pm$ values represent the standard deviation of each metric.

IoTID20 dataset				
Method	Accuracy (%)	F1 (%)	FAR (%)	Latency (ms)
ARF-ADWIN	98.47 $\pm$.26	99.19 $\pm$.14	0.31 $\pm$.11	0.78
ARF-EDDM	98.05 $\pm$.45	98.98 $\pm$.23	0.32 $\pm$.09	1.39
OPA	95.59 $\pm$.18	97.66 $\pm$.10	2.56 $\pm$.13	0.19
SRP	98.83 $\pm$.20	99.38 $\pm$.11	0.31 $\pm$.08	4.03
KNN-ADWIN	95.67 $\pm$.09	97.73 $\pm$.04	1.08 $\pm$.05	0.43
HT	95.82 $\pm$.06	97.82 $\pm$.03	0.38 $\pm$.05	0.6
HATT	96.54 $\pm$.15	98.19 $\pm$.08	0.53 $\pm$.21	0.48
PWPAE	98.94 $\pm$.22	99.44 $\pm$.11	0.21 $\pm$.07	10.72
MSANA	98.81 $\pm$.31	99.37 $\pm$.17	0.08 $\pm$.03	4.72
AEWAE	99.44$\pm$.06	99.70$\pm$.03	0.002$\pm$.006	2.93

other evaluated models when the α is set to 0.253. It achieves the highest accuracy at 99.44% and an F1-score of 99.70%. Furthermore, AEWAE demonstrates exceptional speed with an inference time of just

2.93 ms per packet, surpassing other ensemble methods such as PWPAE and MSANA. The proposed AEWAE model also exhibits exceptional performance in terms of FAR, achieving a rate of 0.002%, which is the lowest compared to all other evaluated methods.

The α value of 0.006 for the NSL-KDD dataset indicates that the model prioritizes recent prediction errors (err) in its updating mechanism. In this scenario, the AEWAE model is tailored to concentrate on the most current error data, thus considering a limited range of historical samples. This suggests that the NSL-KDD dataset may present rapidly changing or non-stationary patterns, where older errors become less relevant for future predictions. Hence, swiftly adapting to new data inputs, reflected by the lower α value, is key to superior performance in this context.

Conversely, in the IoTID20 dataset, the higher α value of 0.253 implies that the model assigns significant importance to historical error data, denoted as cum_err . This means that the model encapsulates a broader span of historical samples, favoring past error information for its learning process. This could reflect that the IoTID20 dataset exhibits

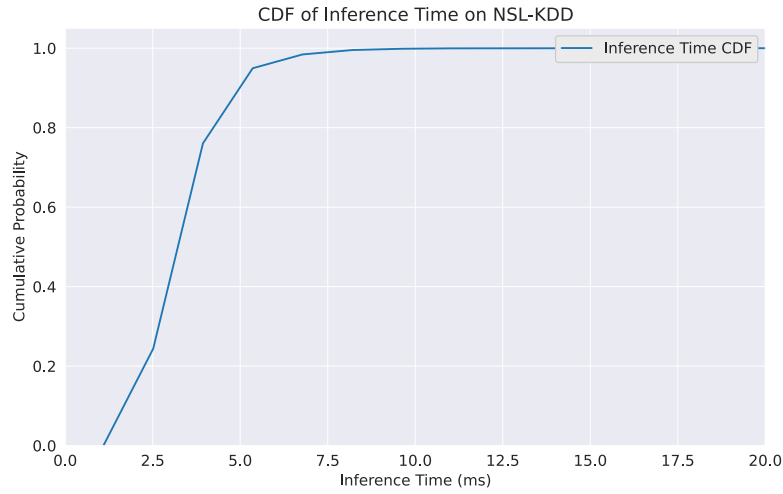


Fig. 6. Cumulative distribution function (CDF) analysis of the processing time for the introduced model on the NSL-KDD dataset.

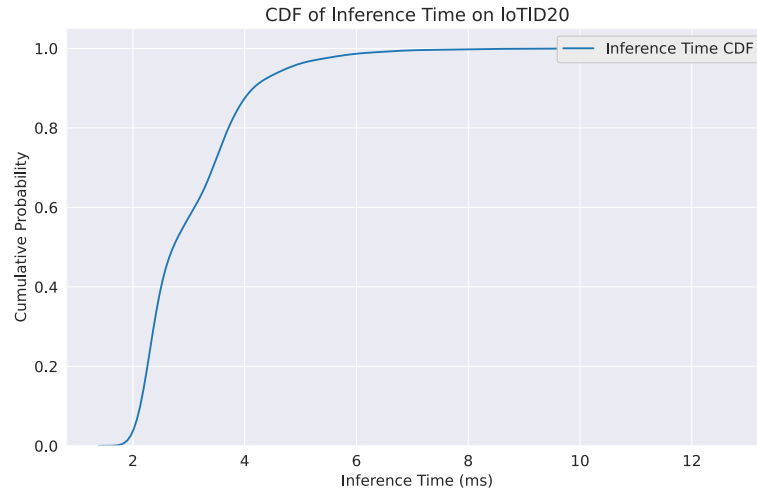


Fig. 7. Cumulative distribution function (CDF) analysis of the processing time for the introduced model on the IoTID20 dataset.

relatively stable or slow-changing patterns, where historical errors hold valuable insights for predicting future outcomes. Consequently, a greater emphasis on past errors, indicated by the larger α value, facilitates superior predictive performance in this dataset.

The Cumulative Distribution Functions (CDFs) detailing the inference time latency for the NSL-KDD and IoTID20 datasets are depicted in Figs. 6 and 7, respectively. The corroborative showcase in Figs. 6 and 7 substantiates that the proposed model was able to process nearly all samples within an impressive timeframe of 9 ms for the NSL-KDD and 8 ms for the IoTID20 dataset, showcasing the efficacy of the introduced model.

Additionally, to comprehensively evaluate the performance of the proposed AEWA model in IoT environments, we conduct experiments on subsets of the IoT23 and CICIOT2023 datasets. The results, illustrated in Figs. 8 and 9, demonstrate that the AEWA model successfully detects nearly all anomalies with no false positives. These findings underscore the model's exceptional accuracy and robustness in IoT anomaly detection, further emphasizing its potential for real-world applications where precise detection is critical.

4.5. Discussion

We systematically summarize our findings and delve into the underlying factors that contribute to our model's better performance compared to other established baseline models, as follows:

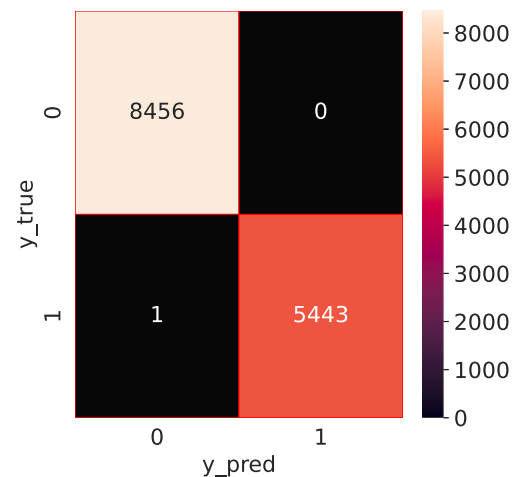


Fig. 8. Confusion matrix for proposed AEWA on the IoT23 dataset.

- **Adaptive weighting with exponentially weighted averages:** AEWA adaptively adjusts model weights using an exponentially weighted moving average based on recent performance, allowing the ensemble to prioritize models performing well and quickly

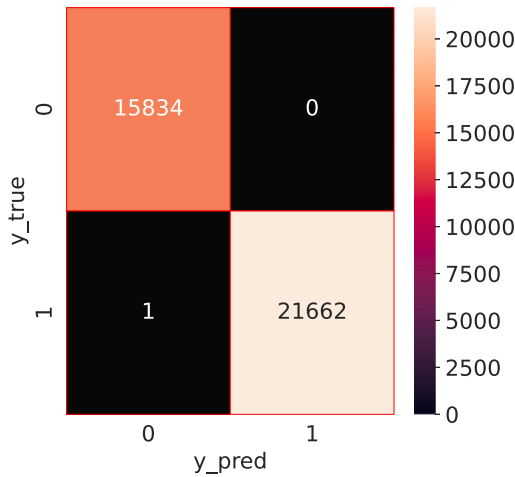


Fig. 9. Confusion matrix for proposed AEWAE on the CICIoT2023 dataset.

respond to concept drift frequent occurrence in IoT data due to environmental changes, user behavior, and system updates.

- **Diverse base models:** The ensemble integrates heterogeneous base models (OP-A, ARF-EDDM, ARF-ADWIN, and KNN-ADWIN), each suited to different concept drifts and data characteristics. This diversity enhances robustness, with ARF models capturing abrupt changes, OPA excelling in online adaptation, and KNN-ADWIN handling non-linear patterns effectively.
- **Recent data emphasis via exponential decay:** The exponential decay factor α prioritizes recent error rates, crucial for IoT data where rapid changes necessitate a focus on the latest observations to maintain accuracy.
- **HPO with PSO:** PSO optimizes α , allowing the model to balance responsiveness to new data with stability, thereby enhancing performance.
- **Computational efficiency:** AEWAE maintains a time complexity of $O(kcb)$, where k is the sample size, c is the class count, and b is the number of base models, ensuring efficiency for real-time processing.
- **Robustness to noise and anomalies:** By dynamically adjusting model weights, AEWAE mitigates noise from individual predictions, smoothing out anomalies and delivering more reliable predictions.

5. Conclusion

In this article, we introduce a drift-adaptive framework designed to address the challenges of concept drift and data imbalance in dynamic IoT data streams. Our proposed framework integrates a comprehensive strategy that includes IoT data preprocessing, foundational model learning, and an innovative online ensembling approach. The architecture initially applies normalization techniques to the data input, followed by the utilization of methods EDDM and ADWIN for the detection of concept drifts. Subsequently, it employs ARF, OPA, and KNN-ADWIN for effective adaptation to the detected drifts. Finally, it utilizes the AEWAE method to ensemble the predictions of the base models. This method stands out by assigning adaptive exponential weights to four base models according to their real-time performance, with a key parameter optimized using Particle Swarm Optimization (PSO) for flexible adjustments under varying conditions. Significantly, our AEWAE framework demonstrates efficient capability in detecting and adapting to concept drifts. Experimental evaluations conducted on the four public datasets demonstrate that AEWAE-based anomaly detection significantly enhances detection accuracy, reduces latency

and FAR. These advancements highlight the efficiency and effectiveness of our framework in facilitating anomaly detection within IoT systems. Looking ahead, we plan to expand the application of our framework to additional domains, especially the Internet of Vehicles (IoV).

CRediT authorship contribution statement

Yafeng Wu: Writing – original draft, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Lan Liu:** Writing – review & editing, Supervision, Resources, Funding acquisition. **Yongjie Yu:** Writing – original draft, Data curation. **Guiming Chen:** Visualization, Investigation. **Junhan Hu:** Validation, Resources.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors extend their heartfelt appreciation to the anonymous reviewers and editors of Applied Soft Computing for their insightful comments and constructive feedback, which have greatly enhanced the quality of this work. Special thanks to Jun Lin (linjun@ceprei.com) for his valuable assistance during the revisions. This research was generously supported by the Guangdong Provincial Education Department's Scientific Research Projects (Specialized in Key Areas) under Grant 2022ZDZX1015.

Data availability

Data will be made available on request.

References

- [1] R. Hassan, F. Qamar, M.K. Hasan, A.H.M. Aman, A.S. Ahmed, Internet of Things and its applications: A comprehensive survey, *Symmetry* 12 (10) (2020) 1674.
- [2] Y. Song, F.R. Yu, L. Zhou, X. Yang, Z. He, Applications of the Internet of things (IoT) in smart logistics: a comprehensive survey, *IEEE Internet Things J.* 8 (6) (2020) 4250–4274.
- [3] L. Chettri, R. Bera, A comprehensive survey on Internet of Things (IoT) toward 5G wireless systems, *IEEE Internet Things J.* 7 (1) (2019) 16–32.
- [4] A.V. Dastjerdi, R. Buyya, Fog computing: Helping the Internet of Things realize its potential, *Computer* 49 (8) (2016) 112–116.
- [5] K. Albulayhi, A.A. Smadi, F.T. Sheldon, R.K. Abercrombie, IoT intrusion detection taxonomy, reference architecture, and analyses, *Sensors* 21 (19) (2021) 6432.
- [6] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, G. Zhang, Learning under concept drift: A review, *IEEE Trans. Knowl. Data Eng.* 31 (12) (2018) 2346–2363.
- [7] I. Ullah, Q.H. Mahmoud, A scheme for generating a dataset for anomalous activity detection in IoT networks, in: *Advances in Artificial Intelligence: 33rd Canadian Conference on Artificial Intelligence, Canadian AI 2020, Ottawa, ON, Canada, May 13–15, 2020, Proceedings 33*, Springer, 2020, pp. 508–520.
- [8] J. Liu, B. Kantarci, C. Adams, Machine learning-driven intrusion detection for Contiki-NG-based IoT networks exposed to NSL-KDD dataset, in: *Proceedings of the 2nd ACM Workshop on Wireless Security and Machine Learning*, 2020, pp. 25–30.
- [9] Y. Liang, N. Vankayalapati, Machine learning and deep learning methods for better anomaly detection in IoT-23 dataset cybersecurity, 2022, Preprint. Available online: <https://github.com/yliang725/Anomaly-Detection-IoT23>. (Accessed 22 December 2022).
- [10] E.C.P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, A.A. Ghorbani, CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment, *Sensors* 23 (13) (2023) 5941.
- [11] R. Ahmad, I. Alsmadi, Machine learning approaches to IoT security: A systematic literature review, *Internet Things* 14 (2021) 100365.
- [12] F. Alwahedi, A. Aldhaheri, M.A. Ferrag, A. Battah, N. Tihanyi, Machine learning techniques for IoT security: Current research and future vision with generative AI and large language models, *Internet Things Cyber- Phys. Syst.* (ISSN: 2667-3452) 4 (2024) 167–185.

- [13] R. Toorajipour, V. Sohrabpour, A. Nazarpour, P. Oghazi, M. Fischl, Artificial intelligence in supply chain management: A systematic literature review, *J. Bus. Res.* 122 (2021) 502–517.
- [14] B. Huang, D. Chaki, A. Bouguettaya, K.-Y. Lam, A survey on conflict detection in iot-based smart homes, *ACM Comput. Surv.* 56 (5) (2023) 1–40.
- [15] X. Wang, H. Zhu, Z. Ning, L. Guo, Y. Zhang, Blockchain intelligence for internet of vehicles: Challenges and solutions, *IEEE Commun. Surv. Tutor.* 25 (4) (2023) 2325–2355, <http://dx.doi.org/10.1109/COMST.2023.3305312>.
- [16] A. Abusitta, G.H. de Carvalho, O.A. Wahab, T. Halabi, B.C. Fung, S. Al Mamoori, Deep learning-enabled anomaly detection for IoT systems, *Internet Things* 21 (2023) 100656.
- [17] H.C. Altunay, Z. Albayrak, A hybrid CNN+ LSTM-based intrusion detection system for industrial IoT networks, *Eng. Sci. Technol. Int. J.* 38 (2023) 101322.
- [18] Z. Wang, J. Li, S. Yang, X. Luo, D. Li, S. Mahmoodi, A lightweight IoT intrusion detection model based on improved BERT-of-Theseus, *Expert Syst. Appl.* 238 (2024) 122045.
- [19] B. Sharma, L. Sharma, C. Lal, S. Roy, Anomaly based network intrusion detection for IoT attacks using deep learning technique, *Comput. Electr. Eng.* 107 (2023) 108626.
- [20] A. Bifet, R. Gavalda, Learning from time-changing data with adaptive windowing, in: *Proceedings of the 2007 SIAM International Conference on Data Mining*, SIAM, 2007, pp. 443–448.
- [21] J. Gama, P. Medas, G. Castillo, P. Rodrigues, Learning with drift detection, in: *Advances in Artificial Intelligence–SBIA 2004: 17th Brazilian Symposium on Artificial Intelligence*, Sao Luis, Maranhao, Brazil, September 29–October 1, 2004. *Proceedings 17*, Springer, 2004, pp. 286–295.
- [22] M. Baena-Garcia, J. del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavalda, R. Morales-Bueno, Early drift detection method, in: *Fourth International Workshop on Knowledge Discovery from Data Streams*, vol. 6, Citeseer, 2006, pp. 77–86.
- [23] P. Domingos, G. Hulten, Mining high-speed data streams, in: *Proceedings of the Sixth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2000, pp. 71–80.
- [24] C. Manapragada, G.I. Webb, M. Salehi, Extremely fast decision tree, in: *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2018, pp. 1953–1962.
- [25] H.M. Gomes, A. Bifet, J. Read, J.P. Barddal, F. Enembreck, B. Pfahringer, G. Holmes, T. Abdessalem, Adaptive random forests for evolving data stream classification, *Mach. Learn.* 106 (2017) 1469–1495.
- [26] H.M. Gomes, J. Read, A. Bifet, Streaming random patches for evolving data stream classification, in: *2019 IEEE International Conference on Data Mining, ICDM, IEEE*, 2019, pp. 240–249.
- [27] V. Losing, B. Hammer, H. Wersing, KNN classifier with self adjusting memory for heterogeneous concept drift, in: *2016 IEEE 16th International Conference on Data Mining, ICDM, IEEE*, 2016, pp. 291–300.
- [28] P.-H. Chiang, S.-C. Tsai, Detection of malicious domains with concept drift using ensemble learning, *IEEE Trans. Netw. Serv. Manag.* (2024) 1, <http://dx.doi.org/10.1109/TNSM.2024.3435516>.
- [29] S. Cai, Y. Zhao, Y. Hu, J. Wu, J. Wu, G. Zhang, C. Zhao, R.N.A. Sosu, CD-BTMSE: A concept drift detection model based on bidirectional temporal convolutional network and multi-stacking ensemble learning, *Knowl.-Based Syst.* 294 (2024) 111681.
- [30] L. Yang, D.M. Manias, A. Shami, Pwpae: An ensemble framework for concept drift adaptation in iot data streams, in: *2021 IEEE Global Communications Conference, GLOBECOM, IEEE*, 2021, pp. 01–06.
- [31] L. Yang, A. Shami, A multi-stage automated online network data stream analytics framework for IIoT systems, *IEEE Trans. Ind. Inf.* 19 (2) (2022) 2107–2116.
- [32] S. Roberts, Control chart tests based on geometric moving averages, *Technometrics* 42 (1) (2000) 97–101.
- [33] J.S. Hunter, The exponentially weighted moving average, *J. Qual. Technol.* 18 (4) (1986) 203–210.
- [34] J. Kennedy, R. Eberhart, Particle swarm optimization, in: *Proceedings of ICNN'95-International Conference on Neural Networks*, vol. 4, IEEE, 1995, pp. 1942–1948.
- [35] J. Montiel, M. Halford, S.M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H.M. Gomes, J. Read, T. Abdessalem, et al., River: machine learning for streaming data in python, *J. Mach. Learn. Res.* 22 (1) (2021) 4945–4952.
- [36] K. Crammer, O. Dekel, J. Keshet, S. Shalev-Shwartz, Y. Singer, Online passive-aggressive algorithms, *J. Mach. Learn. Res.* 7 (19) (2006) 551–585.